DEEP RESEARCH / 2026

Molokh
Технический стек закрытого MVP

Простая и прозрачная архитектура для Web, iOS и Android.
Один разработчик. Один пилотный зал. Минимальный бюджет.

РЕКОМЕНДАЦИЯ

Один универсальный Expo-клиент.
Один Fastify-монолит.
Одна PostgreSQL.

Основано на официальной документации платформ, стандартов и инфраструктуры. Версии проверены на 4 сентября 2026 года.

Содержание

Навигация отражает логику решения: сначала выбор, затем обоснование, реализация, риски и источники.

Область исследования 4

Допущения 4

Прямой ответ 4

Рекомендуемый стек в одной таблице . . . 5

Что меняется относительно прежнего baseline 5

Убрать отдельный Next.js Admin на старте . . . 5

Убрать NestJS на старте . . . 6

Почему один Expo-клиент — оптимальный выбор 6

Что именно должно быть общим . . . 6

Что не нужно насильно объединять . . . 6

Почему не Flutter, Capacitor или Kotlin Multiplatform . . . 6

Архитектура без лишних конструкций 7

Границы процессов . . . 7

Рекомендуемая структура репозитория . . . 7

Backend: явные модули вместо framework-магии 7

Контракты API 8

PostgreSQL — часть Core, а не просто хранилище 8

Почему Kysely, а не тяжёлый ORM . . . 9

Jobs, timers и outbox 9

Realtime: SSE как подсказка, HTTP как команда 9

Состояние клиента и UX-простота 9

Аутентификация и сессии 10

Тестовая стратегия: проверять риск, а не проценты покрытия 10

1. Pure Core tests — Vitest . . . 10

2. PostgreSQL integration — Testcontainers . . . 10

3. API tests — Fastify inject . . . 11

4. Web E2E — Playwright . . . 11

5. Native smoke — Maestro . . . 11

Логи и наблюдаемость **11**

Развёртывание и стоимость сложности **11**

Local и CI . . . 11

Закрытая alpha . . . 11

Живой пилот . . . 12

Политика версий **12**

Чего сознательно не будет в MVP **13**

План внедрения **13**

Этап 0 — ADR и spike, 2-3 дня . . . 13

Этап 1 — фундамент, 4-6 дней . . . 13

Этап 2 — первый vertical slice, 2-3 недели . . . 13

Этап 3 — competition slice, 2-3 недели . . . 13

Этап 4 — пилотная готовность, 1 неделя . . . 14

Триггеры пересмотра **14**

Основные риски и контроль **14**

Итоговая рекомендация **15**

Ограничения исследования **15**

Источники **15**

Реестр ключевых утверждений и источников **17**

Область исследования

Исследование отвечает на один прикладной вопрос: какой минимальный технический стек позволит одному разработчику быстро собрать и безопасно поддерживать Molokh на Web, iOS и Android, не дублируя бизнес-логику и не скрывая сложность Core Engine за чрезмерными фреймворками.

В исследование включены клиентская платформа, backend, API-контракты, база данных, фоновые задания, realtime, авторизация, тестирование, наблюдаемость, доставка и размещение. Визуальный дизайн и маркетинговый сайт сознательно не проектируются: по решению владельца дизайн будет выполняться после доказательства механики.

Допущения

- Закрытый MVP запускается в одном пилотном зале, в одном городе и с ограниченным числом пользователей.
- Разработчик один, работает вместе с AI-ассистентом; бюджет отсутствует или очень мал.
- Athlete-интерфейс должен быть доступен на Web, iOS и Android. Минимальные функции Coach, Gym Admin и Central также должны работать через браузер.
- Сервер остаётся единственным источником истины для прав, статусов, дедлайнов, очередей, Result, Appeal и `allowed_actions`.
- Core Engine уже существенно формализован: риск проекта лежит не в CRUD, а в конкурентных переходах, временных правилах, FIFO, бронированиях и компенсациях.
- Под AOS в запросе понимается Android OS.
- Версии и доступность облачных сервисов проверены на дату отчёта; перед фактическим созданием репозитория их нужно повторно зафиксировать lock-файлом.

Прямой ответ

Для Molokh следует выбрать TypeScript-монорепозиторий, один универсальный Expo-клиент для Web/iOS/Android и один модульный Fastify-монолит с PostgreSQL 18 и отдельным Graphile Worker.

Это не «самый модный» стек, а наименьшая система, которая сохраняет критически важные свойства проекта:

1. один язык и одна предметная терминология во всём репозитории;
2. один пользовательский клиент вместо трёх приложений;
3. одна транзакционная база для Booking, FIFO, Status, Result и Appeal;
4. явные команды и переходы вместо универсального workflow-engine;
5. отсутствие Redis, Kafka, Kubernetes и микросервисов на пилоте;
6. web-адаптация там, где она нужна, без требования «100% одинакового UI»;
7. серверная авторизация через `allowed_actions`, а не повторение правил в приложении.

Рекомендуемый стек в одной таблице

Слой	Решение для MVP	Почему это подходит Molokh
Язык и runtime	TypeScript 5.9, Node.js 24 LTS	Один язык; зрелый LTS; без риска свежего переходного TS 6.0
Репозиторий	pnpm workspaces, без Nx/Turborepo	Встроенных workspace-команд достаточно одному разработчику
Клиент	Expo SDK 57, React Native 0.86, Expo Router	Один проект для Web/iOS/Android; нативные приложения и web из общей кодовой базы
UI	React Native primitives + `StyleSheet` + небольшие design tokens	Минимум зависимостей и предсказуемое платформенное поведение
Server state	TanStack Query	Единая модель fetch/cache/refetch для React DOM и React Native
API	Fastify 5, REST/JSON, command/query endpoints	Нативная JSON Schema-валидация, инкапсуляция модулей, простой тестовый injection
Контракты	TypeBox + OpenAPI 3.1	Одна схема даёт runtime JSON Schema и статический TypeScript-тип
Core	Чистые TypeScript-функции и явный transition registry	Правила можно тестировать без HTTP и базы; нет скрытого workflow DSL
SQL	`pg` + Kysely + нумерованные SQL migrations	Типизированные запросы без потери прямого доступа к locks/constraints
База	PostgreSQL 18	Транзакции, row locks, exclusion constraints, UUIDv7, диапазоны времени
Jobs	Graphile Worker + transactional outbox	Очередь в той же PostgreSQL; не нужен Redis
Realtime	HTTP-команды + SSE hints; refetch после reconnect	Проще WebSocket; realtime не становится источником истины
Auth	Phone OTP adapter; short access token; rotating opaque refresh session	Соответствует phone-first продукту и не отдаёт Core-идентичность внешнему SaaS
Тесты	Vitest + Fastify inject + Testcontainers/PostgreSQL + Playwright + минимальный Maestro smoke	Проверяет чистые правила, реальные транзакции, web и нативный happy path
Доставка	Docker Compose + Caddy; EAS internal builds; статический web	Один серверный контур, автоматический HTTPS, простая раздача пилотных сборок
Наблюдаемость	Pino JSON logs, health/readiness, OTel traces/metrics по мере необходимости	Диагностика без самостоятельного ELK/Grafana-кластера

Что меняется относительно прежнего baseline

Предыдущий implementation pack предлагал отдельный Next.js для Central/Gym Admin и NestJS поверх Fastify. Исследование рекомендует не включать их в стартовый MVP.

Убрать отдельный Next.js Admin на старте

В пилоте административные функции немногочисленны: увидеть pending Result, подтвердить или исправить действие, обработать Appeal/Case, управлять слотами и ролями. Их можно реализовать защищёнными responsive routes того же Expo-приложения с web-специфичными компонентами для таблиц. Expo официально поддерживает универсальный web и допускает как React Native for Web, так и web-only DOM-компоненты. [S3][S4]

Отдельный Next.js нужно извлечь только когда одновременно появятся постоянная Central-команда и действительно плотные desktop-сценарии: массовые таблицы, сложные фильтры, экспорт, многооконная работа или независимый цикл релизов.

Убрать NestJS на старте

Fastify уже предоставляет модульную инкапсуляцию plugins, hooks, нативную JSON Schema-валидацию и сериализацию, type providers, структурированные Pino-логи и HTTP injection для тестов. [S11][S12][S13][S14]

NestJS добавляет DI-контейнер, decorators и собственные соглашения. Для большой команды они могут стандартизировать код, но у одного разработчика увеличивают число слоёв между command handler и транзакцией. Core Molokh лучше защищает не каркас контроллеров, а явная композиция модулей, ограничения PostgreSQL и проверяемые state transitions.

Почему один Expo-клиент — оптимальный выбор

React Native рекомендует начинать новые приложения с framework, а Expo прямо указан как production-grade framework с поддержкой Android, iOS и Web. [S1] Expo позволяет собирать универсальное web-приложение и при необходимости помещать web-only DOM-компоненты рядом с общими React Native-компонентами. [S3] Expo Router создаёт общий file-based routing для universal React Native. [S4]

Что именно должно быть общим

- route map, session/auth lifecycle и deep links;
- API client, TypeBox-контракты и server-state cache;
- экраны Discovery, Match, Chat, Slot, Challenge, Result и Profile;
- design tokens, тексты действий и коды ошибок;
- аналитические события и feature flags;
- accessibility-семантика и тестовые идентификаторы.

Что не нужно насильно объединять

- mobile bottom tabs и wide-screen sidebar;
- swipe-жест и клавиатурная/мышиная альтернатива на web;
- web-таблицы Central и компактные mobile cards;
- web cookie storage и native SecureStore;
- platform-specific push/deep-link adapters.

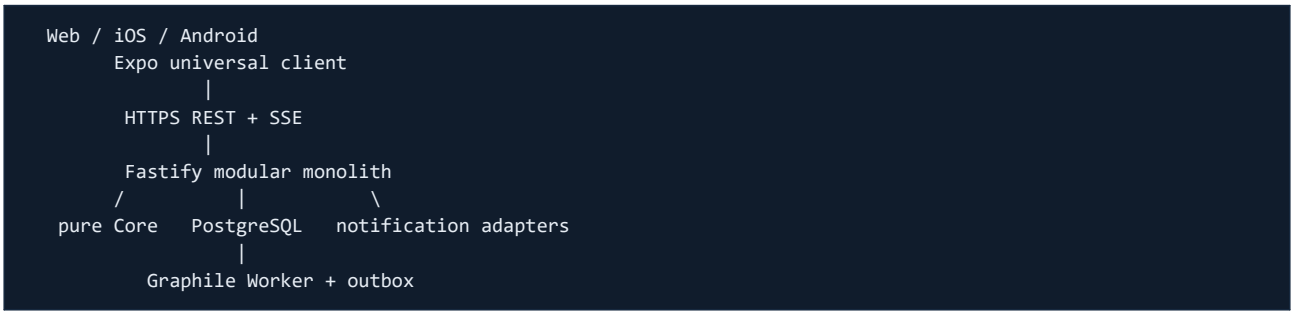
React Native специально предусматривает `Platform` и расширения `.ios`, `.android`, `.native` и web-варианты. Это означает, что универсальность — общая модель и большая часть компонентов, а не искусственный KPI процента переиспользования. [S2]

Почему не Flutter, Capacitor или Kotlin Multiplatform

Альтернатива	Сильная сторона	Почему не первая для Molokh
Flutter	Единый выразительный UI и сильный mobile	Добавляет Dart; официальный FAQ отмечает ограничения document-like web и SEO; нет выгоды единого языка с backend [S5]
Capacitor	Отличен для web-first команды и существующего web UI	Это web-first native container; Molokh mobile-first, с жестами, push и нативными ожиданиями [S6]
Kotlin Multiplatform	Сильное общее ядро Android/iOS	Compose Web/Wasm всё ещё Beta; появляется Kotlin/JVM-экосистема рядом с TypeScript [S7]
Отдельные Swift/Kotlin/Web	Максимальная платформенная свобода	Три кодовые базы и три цикла тестирования несовместимы с одним разработчиком и нулевым бюджетом

Flutter остаётся разумным вторым выбором, если разработчик уже значительно сильнее в Dart, чем в TypeScript. В текущих условиях преимущество одного языка на клиенте, сервере, контрактах и тестах перевешивает различия UI-frameworks.

Архитектура без лишних конструкций



Границы процессов

На пилоте достаточно трёх deployable units:

1. `client-web` — статический export Expo, который отдаёт Caddy;
2. `api` — Fastify process;
3. `worker` — Graphile Worker process.

iOS и Android — сборки того же `apps/client`. API и worker используют одни `packages/core`, `packages/contracts` и `packages/db`, но запускаются отдельно. Worker можно временно запускать в одном контейнере с API только на локальной разработке; в staging и production процессы лучше разделить, чтобы зависшие jobs не влияли на HTTP.

Рекомендуемая структура репозитория

```
apps/
  client/        # Expo: Web, iOS, Android, все роли
  api/           # Fastify routes и command/query handlers
  worker/        # Graphile Worker tasks
packages/
  contracts/     # TypeBox schemas, error codes, OpenAPI
  core/          # state transitions, policies, value objects
  db/            # Kysely types, repositories, SQL migrations
  testkit/       # fixtures, clocks, builders, concurrency helpers
infra/
  compose.yaml
  compose.production.yaml
  Caddyfile
```

Не создавать отдельный `packages/ui`, пока нет второго реального потребителя. Tokens и primitives сначала живут внутри `apps/client`. Не создавать `shared` или `common`: такие папки быстро становятся неясным складом зависимостей.

Backend: явные модули вместо framework-магии

Каждый бизнес-модуль Fastify регистрируется plugin-функцией и имеет одинаковую простую форму:

```
competition/
  routes.ts
  commands/
  queries/
  policy.ts
  repository.ts
  events.ts
```

Типовой command handler:

1. аутентифицирует actor и scope;
2. открывает транзакцию;
3. загружает aggregate с ожидаемой версией и нужным row lock;
4. вызывает чистую Core-функцию перехода;
5. сохраняет aggregate, event, audit и outbox в одной транзакции;
6. возвращает новую projection, `allowed_actions` и стабильный error code.

Запрещены универсальный workflow-engine, decorator-based CQRS bus, динамическая rule DSL и «умные» repository base classes. Они скрывают порядок проверок, а Molokh требует, чтобы спорный результат можно было объяснить человеку через transition ID, actor, policy version и audit trail.

Контракты API

Fastify использует JSON Schema для validation и serialization; его type providers поддерживают TypeBox. TypeBox создаёт JSON Schema, из которой одновременно выводится статический TypeScript-тип. [S12][S13][S15] `@fastify/swagger` генерирует OpenAPI из route schemas и совместим с Fastify 5. [S16]

Рекомендация:

- request/response/error schemas объявлять в `packages/contracts`;
- подключать их к Fastify routes без ручного дублирования DTO;
- генерировать OpenAPI 3.1 в CI и проверять diff;
- импортировать статические типы в Expo-клиент;
- валидировать на клиенте только внешние/сохранённые payloads, а не каждое внутреннее присваивание;
- domain types не делать тождественными transport DTO, если их инварианты различаются.

OpenAPI — язык-независимое описание HTTP API и подходит для документации, генерации клиентов и тестирования, но в TypeScript-монорепозитории не следует генерировать тяжёлый SDK на каждом изменении. Достаточно маленького typed fetch wrapper и схем как source of truth. [S17]

PostgreSQL — часть Core, а не просто хранилище

PostgreSQL 18 подходит проекту из-за строгих транзакций и механизмов, которые непосредственно закрывают инварианты Molokh. Row-level `FOR UPDATE` блокирует конкурирующие изменения выбранных строк до конца транзакции; exclusion constraints предотвращают запрещённые пересечения диапазонов. [S18][S19]

Примеры распределения ответственности:

Инвариант	Где обеспечивать
Один активный status holder	partial unique index + transaction
Два Booking не занимают один Slot	unique/exclusion constraint + row lock
Временные интервалы Coach не пересекаются	range type + GiST exclusion constraint
Один effective Result	unique key + immutable journal
FIFO активирует один worker	ordered selection + competition row lock
Повтор HTTP-команды безопасен	idempotency key + stored response/effect
Event и notification не расходятся	aggregate/event/outbox commit в одной transaction

PostgreSQL 18 добавляет встроенный `uuidv7()`, timestamp-ordered identifiers и расширенные temporal constraints. [S20] Yandex Managed PostgreSQL уже поддерживает версию 18 и называет её рекомендованной для новых проектов в первом году lifecycle. [S21][S22]

Почему Kysely, а не тяжёлый ORM

Kysely даёт type-safe SQL query builder и не отнимает у проекта прямой SQL. Это важно для `FOR UPDATE`, `SKIP LOCKED`, range types, exclusion constraints, partial indexes и `RETURNING`. [S23]

Миграции должны быть нумерованными `.sql` файлами, forward-only в production. Kysely допустим для построения обычных запросов; критические locks и constraints лучше писать явным SQL и покрывать integration tests.

Jobs, timers и outbox

Graphile Worker хранит jobs в PostgreSQL, гарантирует at-least-once execution, поддерживает retries, `SKIP LOCKED` и unique `job_key`. Его собственная документация прямо связывает выбор PostgreSQL queue с уменьшением эксплуатационной сложности небольшой команды. [S24]

Правила:

- каждый job содержит aggregate ID, expected state/version, `deadline_at`, policy version и idempotency key;
- любой handler повторяем и идемпотентен;
- stale timer ничего не меняет, а пишет diagnostic metric/audit;
- push/SMS/email никогда не находятся в бизнес-транзакции;
- outbox consumer хранит delivery receipt и повторяет отправку;
- Redis не добавляется, пока PostgreSQL jobs не покажут измеренную проблему.

Realtime: SSE как подсказка, HTTP как команда

Для MVP не нужен постоянный двусторонний WebSocket protocol. Все изменения выполняются обычными идемпотентными HTTP-командами. SSE сообщает клиенту только `entity_changed`, unread count или необходимость refetch. Expo `fetch` поддерживает единый streaming API на web и mobile, включая `text/event-stream`. [S25]

После reconnect клиент всегда запрашивает canonical projection. Если SSE окажется нестабильным в конкретном background-режиме ОС, correctness не ломается: foreground refetch и push notification восстанавливают состояние.

Перейти на WebSocket стоит только при измеренной потребности: высокая частота chat presence/typing, bidirectional live events или неприемлемая стоимость HTTP+SSE. Сам чат при текущем масштабе отправляет сообщения через HTTP POST.

Состояние клиента и UX-простота

TanStack Query работает и с React DOM, и с React Native; использовать его для server state, cache invalidation, retry и reconnect refetch. [S26] Локальное состояние экрана — `useState`/`useReducer`; Context — только для session, theme и небольших стабильных dependencies.

Не добавлять Redux или Zustand заранее. Добавить отдельный store только после появления состояния, которое одновременно:

- не является серверным;
- живёт дольше route;
- используется многими несвязанными экранами;
- стало болезненно передавать через Context.

Для форм использовать простые controlled inputs; React Hook Form подключать только на длинных административных формах. На пилоте большинство действий Molokh — одна понятная команда с подтверждением, а не большие формы.

Аутентификация и сессии

Вход phone-first реализовать через узкий OTP adapter. SMS — только если пользователь выбрал этот канал; архитектура не должна зависеть от одного поставщика. Верификация профиля и право на Challenge остаются отдельными domain states, а не свойствами auth-провайдера.

Сессия:

- short-lived access token;
- opaque refresh token, в базе хранится только hash;
- новый refresh token при каждом refresh; повтор старого отзывает session family;
- web хранит refresh в `Secure`, `HttpOnly`, `SameSite` cookie;
- native хранит refresh в Expo SecureStore;
- logout/recovery/security case отзывает session family;
- OTP endpoints имеют rate limits по телефону, IP, device и time window.

RFC 9700 требует для public clients sender-constrained refresh tokens или rotation; для MVP rotation проще и прозрачнее. [S27] Expo SecureStore шифрует локальные значения через Android Keystore и iOS Keychain, но не должен быть источником истины для невосстановимых данных. [S28]

Не выбирать Auth0, Clerk, Supabase Auth или Firebase Auth как canonical identity на старте. Внешний сервис допустим как сменный transport/OTP adapter, но Molokh уже имеет собственные verification, recovery, duplicate-account и enforcement workflows.

Тестовая стратегия: проверять риск, а не проценты покрытия

1. Pure Core tests — Vitest

Каждый transition получает tests для allowed/denied paths, boundary time, policy version и deterministic effects. Property/model tests постепенно проверяют запрещённые комбинации, но не заменяют читаемые сценарии.

2. PostgreSQL integration — Testcontainers

Тесты запускают реальный PostgreSQL той же major version и проверяют locks, unique/exclusion constraints, idempotency, outbox atomicity и миграции. Testcontainers предоставляет программно запускаемый PostgreSQL container для Node tests. [S29]

Обязательные race tests:

- два пользователя одновременно claim одного Slot;
- два workers одновременно активируют FIFO head;
- timeout сталкивается с пользовательской командой;
- duplicate Result приходит повторно;
- Appeal correction сталкивается с новым Challenge;
- повтор outbox delivery не создаёт вторичное событие.

3. API tests — Fastify inject

Fastify умеет fake HTTP injection без поднятия сетевого порта; этим покрываются auth, schema validation, stable errors и command responses. [S14]

4. Web E2E — Playwright

Минимальный набор critical journeys проходит Chromium, WebKit и Firefox. Playwright поддерживает эти browser engines. [S30]

5. Native smoke — Maestro

После первого vertical slice добавить только 3-5 smoke flows на Android/iOS: login, Match-to-Booking, Coach Result, Challenge, Appeal entry. Maestro официально поддерживает React Native и Expo/EAS. [S31]

Не переносить все 213 acceptance scenarios в E2E. Большинство должны стать быстрыми Core/API tests; E2E проверяет лишь сквозное соединение экранов и сервисов.

Логи и наблюдаемость

Fastify использует Pino и добавляет request ID; Pino поддерживает redaction. [S11] В production включить JSON logs и исключить phone, OTP, tokens, message text, evidence и birth date.

Минимум до пилота:

- `/health/live` и `/health/ready`;
- `request_id`, `correlation_id`, `causation_id`, actor ID и aggregate ID;
- metrics: overdue jobs, outbox delay, slot conflicts, stuck challenges, pending results, Appeal SLA;
- error tracking для client и server;
- ежедневная проверка backup restore на отдельной test database или по расписанию;
- простой внешний uptime check.

OpenTelemetry оставить как vendor-neutral instrumentation API, но не разворачивать собственный Collector/Grafana/Tempo до появления потребности. Сначала достаточно Pino, health endpoints и нескольких business metrics. [S32]

Развёртывание и стоимость сложности

Local и CI

Docker Compose: PostgreSQL 18, API и worker. Web запускается через Expo dev server. Object storage отсутствует, пока Result media отключены и Central принимает материалы вне приложения.

Закрытая alpha

Один Linux VM в российском регионе:

- Caddy;
- web static export;
- API container;
- worker container;
- PostgreSQL container только для короткой alpha;
- ежедневный encrypted backup во внешнее хранилище.

Docker документирует Compose как применимый к production single-server deployment, а Caddy автоматически получает и обновляет HTTPS-сертификаты. [S33][S34]

Живой пилот

Сохранить один VM для Caddy/API/worker, но вынести PostgreSQL в Managed Service и включить резервное копирование. Yandex Managed PostgreSQL поддерживает PostgreSQL 18 и заявляет защиту инфраструктуры по 152-ФЗ, ISO и другим стандартам; это характеристика провайдера, а не автоматическая юридическая гарантия для приложения. [S21]

Web публикуется как статический Expo export. Android распространяется internal APK/Google Play beta; iOS — TestFlight или ad hoc. EAS Build умеет создавать internal builds и shareable install links, но является опциональным сервисом: код и backend не зависят от EAS. [S35]

Не фиксировать облако в domain code. Все provider-specific элементы — DNS, object storage, push/SMS и deployment scripts — находятся за adapters или infra-конфигурацией.

Политика версий

На дату отчёта актуальный стабильный Expo SDK 57 использует React Native 0.86, React 19.2.3, React Native Web 0.21 и требует Node.js не ниже 22.13. [S8] Node.js 24 находится в LTS, тогда как Node.js 26 ещё Current; production Node должен быть LTS. [S9]

Рекомендация при scaffold:

- Node.js 24 LTS, зафиксированный `.nvmrc`/Volta и Docker image;
- Expo SDK 57 stable; никаких beta/canary packages;
- TypeScript 5.9 на первом slice;
- PostgreSQL 18.x одинаковой major version local/staging/production;
- Fastify 5 и совместимые major versions plugins;
- `packageManager` и lockfile в Git;
- exact/tilde versions для infrastructure-critical packages, weekly dependency review.

TypeScript 6.0 вышел непосредственно перед отчётом и официально описан как significant transition release с breaking changes/deprecations перед TypeScript 7.0. [S10] Начинать пилот на 5.9 безопаснее; переход на 6.0 делать отдельным PR после подтверждения Expo/tooling compatibility.

Чего сознательно не будет в MVP

- отдельного Next.js Admin;
- NestJS, generic CQRS bus и workflow-engine;
- микросервисов, Kafka/NATS и distributed transactions;
- Redis как canonical cache, lock или timer store;
- GraphQL;
- offline business commands;
- optimistic status/result transitions;
- full event sourcing всех сущностей;
- Kubernetes;
- ML matching и скрытого reliability score;
- in-app payments;
- хранения Result media и identity documents;
- сложной design system;
- самостоятельного observability-кластера.

Каждое исключение уменьшает количество мест, где может скрыться несогласованность. В Molokh простота означает не меньше правил, а меньше технических механизмов, реализующих одно и то же правило.

План внедрения

Этап 0 — ADR и spike, 2-3 дня

- Зафиксировать этот стек в коротких ADR.
- Создать pnpm workspace и три apps.
- Проверить один route Expo на Web/iOS/Android.
- Проверить Fastify route с TypeBox и OpenAPI.
- Проверить PostgreSQL migration, transaction и Graphile test job.
- Зафиксировать версии только после зелёного spike.

Этап 1 — фундамент, 4-6 дней

- Session/auth, actor/scope, stable error envelope.
- Transaction helper, idempotency, event/audit/outbox tables.
- Pino redaction, health endpoints, test clock.
- CI: typecheck, unit, integration, web build.

Этап 2 — первый vertical slice, 2-3 недели

```
User -> Community -> Discovery -> Like/Match -> Chat -> Slot -> Booking -> READY_FOR_COACH -> Coach Result -> completed training
```

Сначала Core tests и contracts, затем один end-to-end screen path. Не строить все административные экраны заранее.

Этап 3 — competition slice, 2-3 недели

```
Challenge -> acceptance -> Booking -> Result -> provisional status -> Appeal/finalization
```

Перед UI обязательны race tests для Slot, FIFO, Result и timer conflicts.

Этап 4 — пилотная готовность, 1 неделя

- role-adaptive web screens для Coach/Gym/Central;
- internal Android/iOS builds;
- staging rehearsal, backup/restore drill;
- security checklist, pilot accounts, incident/runbook;
- 5-10 реальных сценариев в зале без изменения Core вручную.

Сроки — ориентир, не обещание: точность зависит от текущего состояния кода и уровня разработчика. Правильный критерий перехода между этапами — выполненный gate, а не календарная дата.

Триггеры пересмотра

Добавить	Только когда наблюдается
Отдельный Next.js Admin	Central работает ежедневно с массовыми таблицами/экспортом или нужен отдельный release cycle
WebSocket	SSE+HTTP не выдерживает измеренную частоту realtime или нужен bidirectional presence
Redis	PostgreSQL jobs/cache создают измеримый bottleneck, который не решается индексами/настройкой
Отдельный search engine	PostgreSQL search перестал выполнять SLO на фактическом объёме
Микросервис	Модуль требует независимого scaling/release/ownership и граница подтверждена нагрузкой
Kubernetes	Несколько сервисов, несколько окружений и deployment frequency делают Compose операционно дороже
Сторонний Identity Provider	Enterprise/social federation стала продуктовой необходимостью
Object storage	Включено хранение evidence/media в продукте
Второе Community на пользователя	Pilot подтвердил multi-community demand и согласованы cross-community policies

Основные риски и контроль

Риск	Вероятность/влияние	Контроль
Универсальный web выглядит как растянутый mobile	Средняя/среднее	responsive shell и web-specific компоненты, а не отдельное приложение
Core протекает в UI	Высокая/высокое	только server `allowed_actions`, stable errors, no client eligibility logic
Race condition ломает статус или слот	Средняя/критическое	DB constraints, lock ordering, Testcontainers race tests
At-least-once job повторяет эффект	Средняя/высокое	idempotency key, inbox/outbox receipts, handler replay tests
Solo developer строит платформу вместо продукта	Высокая/высокое	vertical slices, явный список «не будет в MVP», gate-based план
Expo/web библиотека имеет platform gap	Средняя/среднее	spike на трёх платформах до закрепления зависимости; `.web` adapter
Свежая версия ломает tooling	Средняя/среднее	stable channels, lockfile, Node LTS, TS 5.9, отдельные upgrade PR
Облако становится vendor lock-in	Низкая/среднее	стандартные PostgreSQL/S3/Docker/OpenAPI; provider adapters

Итоговая рекомендация

Строить Molokh как один продукт, один Core и одну транзакционную систему, а не как набор платформенных приложений. Expo даёт общий путь на Web/iOS/Android; Fastify оставляет серверные действия видимыми; PostgreSQL реализует конкуренцию атомарно; Graphile Worker добавляет timers без второго datastore.

Главная инженерная дисциплина — не количество фреймворков, а строгая граница: клиент показывает состояние и разрешённые действия, сервер принимает команды, Core вычисляет переход, PostgreSQL не позволяет нарушить инвариант, audit объясняет результат.

Следующее практическое действие: оформить 8-10 коротких ADR и выполнить трёхдневный technical spike. После зелёного spike можно обновить `implementation/ARCHITECTURE.md` и `implementation/README.md`, затем начинать фундамент первого vertical slice.

Ограничения исследования

- Не выполнялись benchmark или prototype на реальном коде Molokh; производительность оценивается по архитектурному профилю пилота.
- Не рассчитана точная стоимость облака: она зависит от региона, скидок, storage и выбранного SLA; перед запуском нужен расчёт в актуальном калькуляторе.
- Отчёт не является юридическим заключением о персональных данных, возрастных ограничениях или спортивной ответственности.
- App Store/Google Play rules, версии SDK и cloud offerings меняются; перед публикацией их нужно перепроверить.
- Рекомендация оптимизирована под одного разработчика и один зал; при другой команде или масштабе решение может измениться по указанным триггерам.

Источники

- [S1] React Native, Get Started / framework recommendation: https://reactnative.dev/docs/environment-setup
- [S2] React Native, Platform-Specific Code: https://reactnative.dev/docs/platform-specific-code
- [S3] Expo, Develop websites with Expo: https://docs.expo.dev/workflow/web/
- [S4] Expo Router, Introduction: https://docs.expo.dev/router/introduction/
- [S5] Flutter, Web FAQ: https://docs.flutter.dev/platform-integration/web/faq
- [S6] Capacitor, Introduction: https://capacitorjs.com/docs
- [S7] Kotlin Multiplatform, supported platform stability: https://kotlinlang.org/docs/multiplatform/supported-platforms.html
- [S8] Expo SDK reference / version matrix: https://docs.expo.dev/versions/latest/
- [S9] Node.js release status: https://nodejs.org/en/about/previous-releases
- [S10] TypeScript 6.0 release notes: https://www.typescriptlang.org/docs/handbook/release-notes/typescript-6-0.html
- [S11] Fastify logging and Pino: https://fastify.dev/docs/latest/Reference/Logging/
- [S12] Fastify validation and serialization: https://fastify.dev/docs/latest/Reference/Validation-and-Serialization/
- [S13] Fastify type providers: https://fastify.dev/docs/latest/Reference/Type-Providers/
- [S14] Fastify testing and HTTP injection: https://fastify.dev/docs/latest/Guides/Testing/
- [S15] TypeBox repository and design: https://github.com/sinclairzx81/typebox
- [S16] `@fastify/swagger`: https://github.com/fastify/fastify-swagger
- [S17] OpenAPI Specification 3.1: https://spec.openapis.org/oas/v3.1.0.html
- [S18] PostgreSQL 18 explicit locking: https://www.postgresql.org/docs/18/explicit-locking.html
- [S19] PostgreSQL 18 constraints: https://www.postgresql.org/docs/18/ddl-constraints.html
- [S20] PostgreSQL 18 release notes: https://www.postgresql.org/docs/18/release-18.html

- [S21] Yandex Managed PostgreSQL: https://yandex.cloud/en/docs/managed-postgresql/
- [S22] Yandex PostgreSQL version policy: https://yandex.cloud/en/docs/managed-postgresql/concepts/update-policy
- [S23] Kysely documentation: https://kysely.dev/docs/intro
- [S24] Graphile Worker introduction: https://worker.graphile.org/docs
- [S25] Expo `fetch` and streaming: https://docs.expo.dev/versions/latest/sdk/expo/
- [S26] TanStack Query installation and RN compatibility: https://tanstack.com/query/latest/docs/framework/react/installation
- [S27] RFC 9700, OAuth 2.0 Security Best Current Practice: https://www.rfc-editor.org/rfc/rfc9700.html
- [S28] Expo SecureStore: https://docs.expo.dev/versions/latest/sdk/securestore/
- [S29] Testcontainers for Node.js, PostgreSQL: https://node.testcontainers.org/modules/postgresql/
- [S30] Playwright browser support: https://playwright.dev/docs/browsers
- [S31] Maestro React Native support: https://docs.maestro.dev/platform-support/react-native
- [S32] OpenTelemetry JavaScript: https://opentelemetry.io/docs/languages/js/
- [S33] Docker Compose in production: https://docs.docker.com/compose/how-tos/production/
- [S34] Caddy automatic HTTPS: https://caddyserver.com/docs/automatic-https
- [S35] Expo EAS internal distribution: https://docs.expo.dev/build/internal-distribution/

Реестр ключевых утверждений и источников

Утверждение	Основание	Тип вывода
Expo покрывает Android/iOS/Web и рекомендован RN для новых приложений	S1, S3, S4, S8	Прямой факт + архитектурный вывод
Platform-specific UI не противоречит общей кодовой базе	S2, S3	Прямой факт
Flutter/Capacitor/KMP возможны, но хуже соответствуют текущим ограничениям	S5, S6, S7	Сравнительный вывод из официальных ограничений
Fastify достаточно без NestJS	S11-S16	Архитектурный вывод из доступных механизмов
TypeBox уменьшает дублирование schema/type	S13, S15-S17	Прямой факт + рекомендация
PostgreSQL должен обеспечивать критические concurrency invariants	S18-S20	Архитектурный вывод из DB mechanisms и Core Molokh
PostgreSQL 18 доступен в Yandex Cloud	S21, S22	Прямой факт на дату отчёта
Redis не нужен для MVP jobs	S24	Архитектурный вывод, совпадающий с целью Graphile Worker
SSE достаточно как invalidation hint	S25 + server-authoritative Core	Архитектурная рекомендация; требует spike
Refresh rotation нужна для public client	S27	Нормативная security recommendation
SecureStore подходит для native refresh secret	S28	Прямой факт + архитектурный вывод
Реальная PostgreSQL нужна в integration tests	S18-S20, S29	Рекомендация по риску concurrency
Compose+Caddy достаточно single-server pilot	S33, S34	Архитектурный вывод для заданного масштаба
Node 24 LTS и TS 5.9 снижают version risk	S8-S10	Времязависимая рекомендация на 2026-09-04